

Documentazione:

Sistema di Gestione Sedute di Laurea

Di Donato Vincenzo, Guarino Edoardo Manzo Stefano

2026

1. Introduzione e Scelte Architettureali

Il diagramma delle classi modella l'architettura logica (Model) del sistema informativo per la gestione dei tirocini e delle sedute di laurea. L'applicazione è strutturata secondo un'architettura multilivello che separa la logica di dominio, il flusso di controllo e l'interfaccia utente.

L'obiettivo principale è stato separare chiaramente gli "Attori" del sistema (le persone fisiche) dalle "Entità di Dominio" (i documenti e gli eventi gestiti), garantendo la massima tracciabilità dei dati.

2. Architettura dei Package

I package fondamentali analizzati in questo documento sono:

- **model:** Contiene le entità del dominio.
- **controller:** Gestisce il flusso dell'applicazione e la comunicazione tra l'interfaccia utente e il modello.
- **gui:** Contiene le classi dedicate all'Interfaccia Grafica Utente (Graphical User Interface).
- **dao:** Definisce le interfacce astratte per l'accesso ai dati.
- **implementazioneDao:** Contiene le implementazioni concrete delle interfacce DAO che eseguono le query SQL sul database PostgreSQL.
- **database_connection:** Gestisce il livello infrastrutturale a basso livello per l'apertura, la configurazione e il rilascio delle connessioni JDBC.

3. Il Package *model*

Il package model rappresenta il cuore del sistema e implementa le entità reali del dominio.

3.1 La Gerarchia degli Utenti (Ereditarietà)

La scelta progettuale più importante riguarda la gestione degli accessi. Per evitare la duplicazione dei dati anagrafici e delle credenziali, è stata creata una struttura gerarchica.

- Utente (Classe Astratta):
 - Significato: È la "superclasse" radice del sistema. Rappresenta una persona generica registrata sulla piattaforma.
 - Scelta della classe Astratta (<>): È stata resa astratta perché nel dominio applicativo non esiste un utente generico: chiunque acceda è necessariamente o uno Studente o un Docente. Questo impedisce l'istanziamento di utenti senza un ruolo specifico.
 - Scelta degli attributi Protetti (#): Gli attributi anagrafici (id, nome, cognome, email, username, password) sono impostati come protected. Questa è una scelta mirata a favorire le classi figlie: Studente e Docente ereditano questi campi e potranno accedervi direttamente, mantenendoli però nascosti al resto del sistema esterno.
- Studente:
 - Significato: Eredita da Utente e rappresenta il candidato.
 - Attributi: Aggiunge matricola (privato), un dato esclusivo di questo ruolo.
 - Metodi: Le operazioni *richiediTirocinio()* e *caricaTesi()* riflettono le azioni attive che lo studente compie sul sistema, prendendo in input gli oggetti necessari (rispettivamente l'argomento scelto e il file da allegare).
- Docente:
 - Significato: Eredita da Utente. Agisce come proponente di tirocini e relatore delle tesi.
 - Metodi: Contiene la logica di approvazione (*valutaRichiesta*, *valutaTesi*). Restituisce inoltre liste di dati, come *getTirociniInCorso()*.
- Coordinatore:
 - Significato: Eredita direttamente da Docente (che a sua volta eredita da Utente).
 - Scelta progettuale: Questa ereditarietà a cascata modella perfettamente la specifica "Il coordinatore del corso di laurea è anch'esso un docente". Il Coordinatore possiede tutte le capacità di un Docente (può avere tesisti), ma ha in più i privilegi amministrativi per inserire sedute (*inserisciSeduta()*) e generare la lista dei relatori (*formaCommissione()*).

3.2 Le Entità di Dominio

Queste classi rappresentano gli oggetti gestiti dal sistema. I loro attributi sono mantenuti strettamente privati (-) per garantire il completo incapsulamento (i valori verranno letti tramite metodi "get" in fase di implementazione Java).

- **ArgomentoTirocinio:**
 - Significato: Rappresenta la proposta formativa creata da un docente.
 - Attributi: titolo, tipo (Interno/Esterno) e il referenteAziendale. Quest'ultimo sarà valorizzato solo se il tirocinio è di tipo esterno.
- **RichiestaTirocinio:**
 - Significato: È la classe di transizione ("classe associativa" logica) che unisce la volontà dello studente con l'argomento offerto.
 - Scelta progettuale: È fondamentale che esista come classe autonoma perché deve memorizzare il suo ciclo di vita temporale tramite l'attributo stato (In Attesa, Approvata, Rifiutata).
- **Tesi:**
 - Significato: Il prodotto finale del lavoro dello studente.
 - Attributi: Contiene il riferimento al documento fisico caricato (*filePath*) e uno stato indipendente per l'approvazione finale del relatore.
- **SedutaLaurea:**
 - Significato: L'evento conclusivo.
 - Attributi: Raggruppa le informazioni logistiche: data, ora e luogo.

3.3 Le Enumerazioni (<>)

L'uso dei tipi enumerati è una "best practice" ingegneristica per garantire l'integrità dei dati e prevenire anomalie nel database o nel software.

- **Stato (IN_ATTESA, APPROVATA, RIFIUTATA):** Condivisa da **RichiestaTirocinio** e **Tesi**. Impedisce l'inserimento di stati anomali causati da errori di battitura manuali.
- **TipoTirocinio (INTERNO, ESTERNO):** Limita rigorosamente le categorie di tirocini ammesse dalla segreteria.

3.4 Le Relazioni e le Molteplicità (I Collegamenti)

I collegamenti tra le classi descrivono le "regole di business" (vincoli) del sistema gestionale.

- Docente (1) --- (*) ArgomentoTirocinio:
 - Significato: Un singolo docente può proporre infiniti argomenti nel tempo, ma un argomento specifico è inserito da esattamente un docente (che ne diventerà il relatore).
- Studente (1) --- (*) RichiestaTirocinio:
 - Significato: Uno studente può effettuare più richieste. Questa molteplicità (*) è necessaria perché, in caso di rifiuto di una richiesta, lo studente dovrà poterne compilare una nuova, generando uno storico.
- RichiestaTirocinio (*) --- (1) ArgomentoTirocinio:
 - Significato: Molti studenti (tramite le loro richieste) possono competere o candidarsi per lo stesso identico argomento proposto dal docente.
- RichiestaTirocinio (1) --- (0..1) Tesi:
 - Scelta progettuale critica: Questo è il legame logico più raffinato. Una richiesta (se approvata) darà vita ad al massimo una tesi (0..1). Zero (0) rappresenta la fase in cui il tirocinio è in corso; Uno (1) indica che il tirocinio è concluso e la tesi è stata caricata. Questa associazione permette di risalire sempre al docente relatore partendo dalla Tesi.
- Tesi (*) --- (1) SedutaLaurea:
 - Significato: Durante una specifica giornata di seduta di laurea verranno discusse contemporaneamente numerose tesi (*), ma una determinata tesi viene presentata in una sola data esatta.
- Coordinatore (1) --- (*) SedutaLaurea:
 - Significato: Il coordinatore è la singola figura preposta alla creazione e organizzazione del calendario delle sedute.

4. Il Package *controller*

Il package controller funge da intermediario tra l'interfaccia grafica e i dati applicativi (inclusi l'interfacciamento con i layer DAO per la persistenza dei dati). Il sistema implementa un singleton **Controller** che gestisce unicamente la sessione dell'utente loggato, l'istradamento delle operazioni grafiche (es. inoltro del file della tesi o della richiesta tirocinio) e il recupero delle liste aggiornate (es. sedute disponibili o tirocini in corso) dialogando direttamente con le interfacce DAO.

5. Il Package *gui*

Il package gui racchiude l'interfaccia visiva con la quale l'utente interagisce. Tutte le finestre del sistema estendono **JFrame** (Standard Java Swing).

5.1 Finestre Principali e Autenticazione

- **LoginFrame**: La schermata di avvio del sistema. Raccoglie le credenziali dell'utente e indirizza verso la corretta dashboard in base al ruolo (Studente, Docente, Coordinatore).
- **HomeStudente**: Dashboard dedicata allo studente, che abilita i pulsanti e i menu a tendina in base allo stato di avanzamento della sua pratica.
- **HomeDocente / HomeCoordinatore**: Dashboard specifiche che sbloccano funzionalità avanzate a seconda dei privilegi dell'utente loggato.

5.2 Schermate Operative

- **CreaArgomento**: Form per l'inserimento di nuove proposte di tirocinio da parte del Docente.
- **GestisciRichieste / ValutaTesi**: Schermate dedicate al Docente per l'accettazione o il rifiuto delle pratiche inoltrate, con possibilità di visualizzare il file fisico della tesi.
- **PaginaLaurea**: Interfaccia per lo Studente per effettuare l'upload del file e selezionare la seduta.
- **TirociniInCorso**: Schermata di riepilogo per monitorare gli studenti correntemente assegnati.
- **CreaSeduta / GestioneCommissione**: Interfacce esclusive per il Coordinatore per pianificare le date ed estrapolare automaticamente i docenti necessari per la commissione.

6. Il Package *dao*

Il package **dao** (Data Access Object) definisce il livello di astrazione per l'accesso ai dati. Utilizzando questo design pattern, il sistema separa la logica del model dalle specifiche operazioni di persistenza, definendo interfacce che stabiliscono le firme dei metodi senza preoccuparsi dell'implementazione del database sottostante.

- **UtenteDAO**: Interfaccia fondamentale per la gestione delle credenziali; espone metodi per recuperare gli utenti in fase di login e determinarne il ruolo.
- **ArgomentoTirocinioDAO**: Definisce i metodi per l'inserimento di nuovi argomenti proposti dai docenti e per la ricerca/lettura da parte degli studenti.
- **RichiestaTirocinioDAO**: Definisce le operazioni necessarie per gestire il ciclo di vita di una pratica (creazione, aggiornamento dello stato in caso di approvazione/rifiuto e recupero storico).
- **TesiDAO**: Interfaccia che espone i metodi per il salvataggio dei percorsi dei file (filePath) e la conseguente validazione o reiezione da parte del relatore.
- **SedutaLaureaDAO**: Definisce le signature per l'inserimento di nuove date nel calendario accademico e il recupero delle sedute disponibili o dei relativi elenchi di commissione.

7. Il Package *implementazioneDao*

Questo package contiene le implementazioni concrete delle interfacce definite nel package **dao**. Le classi interagiscono direttamente con il database relazionale PostgreSQL, eseguendo le query SQL necessarie.

- **UtentePostgresDao**: Implementa **UtenteDAO**. Responsabile della query di autenticazione interrogando le tabelle del database PostgreSQL.
- **ArgomentoTirocinioPostgresDao**: Implementa **ArgomentoTirocinioDAO**. Esegue istruzioni di INSERT per salvare la proposta e di SELECT per popolare le tendine lato studente.
- **RichiestaTirocinioPostgresDao**: Implementa **RichiestaTirocinioDAO**. Gestisce le operazioni SQL relative al ciclo di vita della richiesta e dell'eventuale motivazione di rifiuto.
- **TesiPostgresDao**: Implementa **TesiDAO**. Traduce le operazioni di approvazione/rifiuto del file in query di UPDATE per modificare lo stato della tupla.
- **SedutaLaureaPostgresDao**: Implementa **SedutaLaureaDAO**. Estrae dal database le aggregazioni e le JOIN necessarie per determinare quali docenti devono presenziare a una determinata seduta in base alle tesi assegnate.

8. Il Package *database_connection*

Il package *database_connection* gestisce il livello più basso dell'infrastruttura dati, occupandosi dell'apertura e della chiusura sicura delle connessioni verso il DBMS.

- **ConnessioneDatabase**: Classe (implementata secondo il pattern Singleton) che si occupa di istanziare e mantenere un'unica connessione JDBC (Java Database Connectivity) verso PostgreSQL. La connessione viene riutilizzata per tutta la durata dell'applicazione; sono le singole implementazioni DAO a essere responsabili della chiusura dei propri Statement e ResultSet per evitare memory leak.
- **Gestione delle transazioni**: Alcune operazioni applicative richiedono che più scritture sul database avvengano in modo atomico. Per evitare stati incoerenti, **ConnessioneDatabase** disattiva temporaneamente l'autocommit, esegue le operazioni richieste e conferma (commit) o annulla (rollback) il blocco nella sua interezza in base all'esito complessivo, ripristinando infine il comportamento normale della connessione.